

Prompt Engineering (PE lab) 1

Unit 1

Main Technologies : "Python, OpenAI API, Transformers, Prompt Engineering "

S.No	Aim / Experiment	Technologies Used
1	Environment & Connectivity: Install required packages (e.g., transformers, openai); securely configure the API key; run a simple "Hello, world" prompt to verify model access.	Python, OpenAI API, Transformers
2	Baseline vs. Enhanced Prompts: Execute a naïve prompt ("Write a one-paragraph bio of Ada Lovelace.") and an enhanced prompt that adds role framing, specificity, and explicit format instructions; compare both outputs for relevance, completeness, and style	LLM, Prompt Engineering
3	Iterative Refinement on a Simple Task: Summarize the plot of the Shakespearean play Romeo and Juliet in two sentences through three rounds of prompt tweaking: a.Minimal instruction b. Addition of length and style constraints c. Specification of key content elements (setting and theme) Document how each iteration changes and improves the result	LLM, Prompt Engineering
4	Diagnosing Prompt Failures & Edge Cases: Craft a vague or contradictory prompt; analyze the failure mode (ambiguity, missing context, or format errors); refine the prompt by adding examples or clarifying instructions.	LLM, Prompt Analysis

Experiment 1

```

import sys
import os

sys.path.append(
    os.path.abspath(
        os.path.join(os.path.dirname(__file__), "..")
    )
)

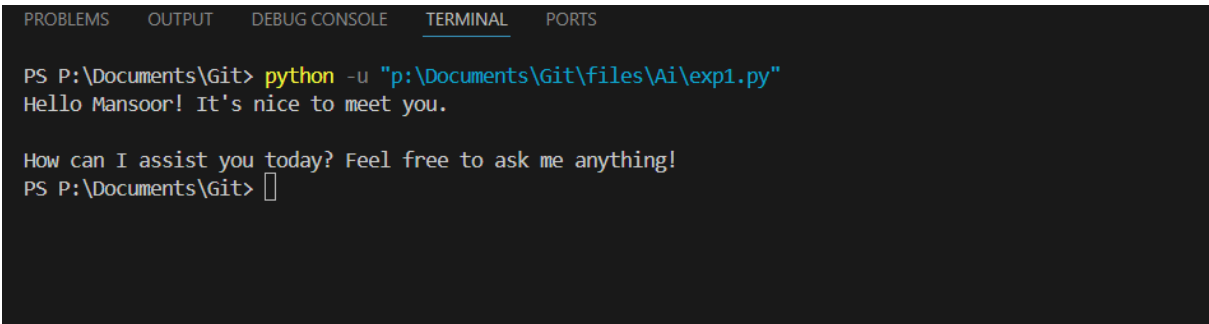
from ai_client import ask_ai

response = ask_ai("Hello, there I am Mansoor")

print(response)
with open("output.txt", "w", encoding="utf-8") as f:
    f.write(response)

```

Output :



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS P:\Documents\Git> python -u "p:\Documents\Git\files\Ai\exp1.py"
Hello Mansoor! It's nice to meet you.

How can I assist you today? Feel free to ask me anything!
PS P:\Documents\Git> 

```

Experiment 2

```

import sys
import os

sys.path.append(
    os.path.abspath(
        os.path.join(os.path.dirname(__file__), "..")
    )
)

```

```

)

from ai_client import ask_ai

# Baseline Prompt
baseline_prompt = "Write a one-paragraph bio of Ada Lovelace."

# Enhanced Prompt
enhanced_prompt = """
You are a professional historian.

Write a one-paragraph biography of Ada Lovelace.

Requirements:
- 120-150 words
- Mention her collaboration with Charles Babbage
- Explain why she is considered the first computer programmer
- Use an academic yet easy-to-read style
- Output exactly one paragraph
"""

print("=" * 60)
print("BASELINE PROMPT")
print("=" * 60)

print(ask_ai(baseline_prompt))

print("\n" + "=" * 60)
print("ENHANCED PROMPT")
print("=" * 60)

print(ask_ai(enhanced_prompt))
with open("output.txt", "w", encoding="utf-8") as f:
    f.write(enhanced_prompt)

```

Output:

Try doing it your own way—you can run the code yourself. (this is not yours outcome)

Experiment 3

```
import sys
import os

sys.path.append(
    os.path.abspath(
        os.path.join(os.path.dirname(__file__), "..")
    )
)

from ai_client import ask_ai

prompts = {
    "ROUND 1: BASIC PROMPT": """
Summarize the plot of Romeo and Juliet in two sentence
s.
""",

    "ROUND 2: LENGTH + STYLE": """
Summarize the plot of Romeo and Juliet in exactly two s
entences.

Requirements:
- 30 to 40 words total
- Use clear and simple language
- Suitable for high school students
""",

    "ROUND 3: CONTENT + SETTING + THEME": """
Summarize the plot of Romeo and Juliet in exactly two s
entences.

Requirements:
- 30 to 40 words total
```

```

        - Use clear and simple language
        - Suitable for high school students
        - Mention that the story takes place in Verona, Italy
        - Mention the theme of love and conflict
    """

}

for title, prompt in prompts.items():
    print("=" * 60)
    print(title)
    print("=" * 60)

    try:
        response = ask_ai(prompt)
        print(response)
    except Exception as e:
        print(f"Error: {e}")

    print()

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(response)

```

Output :

Experiment 4

```

import sys
import os

sys.path.append(
    os.path.abspath(
        os.path.join(os.path.dirname(__file__), "..")
    )
)

from ai_client import ask_ai
# Vague / Contradictory Prompt

```

```

bad_prompt = """
Write a detailed summary of Artificial Intelligence.
Keep it under 20 words and explain everything important.
"""

# Improved Prompt
good_prompt = """
You are an AI educator.

Write a concise summary of Artificial Intelligence.

Requirements:
- 40 to 50 words
- Explain what AI is
- Mention one practical application
- Use simple language suitable for beginners
- Output exactly one paragraph
"""

print("=" * 60)
print("BAD PROMPT")
print("=" * 60)

print(ask_ai(bad_prompt))

print("\n" + "=" * 60)
print("REFINED PROMPT")
print("=" * 60)

print(ask_ai(good_prompt))

with open("output.txt", "w", encoding="utf-8") as f:
    f.write(bad_prompt, good_prompt)

```

Output :

Error & ai_client

Create : ai_client.py

```
from dotenv import load_dotenv
from google import genai
from error_handler import handle_error
import os
import time

load_dotenv()

client = genai.Client(
    api_key=os.getenv("GOOGLE_API_KEY")
)

MODELS = [
    "gemini-2.5-flash",
    "gemini-3.1-flash-lite",
    "gemma-4-31b",
    "gemma-4-26b",
    "gemini-2.5-flash-lite"
]

def ask_ai(prompt):

    for model in MODELS:
        try:
            response = client.models.generate_content(
                model=model,
                contents=prompt
            )

            time.sleep(2)

            return response.text.strip()

        except Exception:
            continue
```

```
return "All models failed."
```

Create : error_handler.py

```
def handle_error(error):
    error_msg = str(error)

    if "429" in error_msg:
        return "ERROR 429: Too Many Requests (Rate Limit Exceeded)"

    elif "API key" in error_msg or "api_key" in error_msg:
        return "Invalid API Key"

    elif "400" in error_msg:
        return "ERROR 400: Bad Request"

    elif "401" in error_msg:
        return "ERROR 401: Unauthorized"

    elif "403" in error_msg:
        return "ERROR 403: Forbidden"

    elif "404" in error_msg:
        return "ERROR 404: Not Found"

    elif "500" in error_msg:
        return "ERROR 500: Internal Server Error"

    elif "502" in error_msg:
        return "ERROR 502: Bad Gateway"

    elif "503" in error_msg:
        return "ERROR 503: Service Unavailable"

    elif "504" in error_msg:
```



```
return "ERROR 504: Gateway Timeout"  
  
return f"Unexpected Error: {error_msg}"
```